



PROVA SUBSTITUTIVA
FASE 4 - FRNT

Thamiris da Silva Adriano

Versão 1

Esta prova substitutiva tem como objetivo consolidar os conhecimentos adquiridos ao longo da Fase 4 - Arquiteturas Avançadas, Clean Architecture, Design System e Desenvolvimento Seguro.

O estudante deverá demonstrar domínio dos conceitos fundamentais de arquitetura front-end moderna, criando uma aplicação estruturada com camadas bem definidas, separação clara de responsabilidades, princípios de Clean Architecture e boas práticas de segurança e escalabilidade.

Problema

Uma empresa está iniciando o desenvolvimento de uma nova plataforma interna chamada SafeExpenses, cujo objetivo é organizar solicitações de reembolso, aprovações e relatórios financeiros.

Atualmente, não existe uma estrutura padronizada de código e cada time desenvolve suas telas e regras de forma independente, causando:

- Falta de padronização visual;
- Baixa reutilização de componentes;
- Dificuldade de manutenção;
- Bugs recorrentes devido a regras duplicadas;
- Riscos de segurança na manipulação dos dados.

A diretoria decidiu que a aplicação deverá ser desenvolvida utilizando princípios de arquitetura limpas, Design System centralizado e práticas de segurança front-end.

Sua missão é construir a primeira versão do sistema com foco na arquitetura, não na quantidade de funcionalidades.

Objetivo

Desenvolver uma aplicação front-end composta por:

Arquitetura em camadas baseada em Clean Architecture, incluindo:

- Presentation;
- Domain;
- Application (use cases);
- Infrastructure (services, gateways, adapters).

Mini Design System avançado, contendo pelo menos:

- Button;
- Input/TextField;
- Card;
- Feedback (Alert/Toast);
- Componente de validação.

Todos seguindo princípios de acessibilidade e tokens de design.

Camada de regras de negócio isolada (Domain Layer)

- Entities;
- Interfaces / contratos;
- Validações puras.

Práticas de desenvolvimento seguro, incluindo:

- Sanitização de entrada;
- Prevenção de XSS;
- Uso correto de storage;
- Isolamento de secrets;
- Validação consistente de dados.

Aplicação mínima funcional com:

- Cadastro de uma despesa;
- Edição de uma despesa;
- Listagem;

- Persistência apenas local (localStorage ou mocks).

A entrega deve demonstrar como a arquitetura foi aplicada — **a robustez da arquitetura é mais importante que o número de features.**

Requisitos do sistema

1. Estrutura obrigatória seguindo Clean Architecture

A aplicação deve ser dividida em camadas:

Domain

- Entities;
- Types/Interfaces;
- Regras puras.

Application

- Use cases (ex.: CreateExpense, UpdateExpense, ListExpenses);
- Orquestração entre domain e infraestrutura.

Infrastructure

- Adaptadores de storage;
- Serviços externos mockados.

Presentation

- Páginas / Componentes;
- Hooks / Controllers;
- Consumo dos use cases.

2. Design System Avançado

O Design System deve conter:

- Tokens de design (cores, tipografia, espaçamentos);
- Componentes reutilizáveis;
- Componentes acessíveis;
- Estrutura documentada no README.

3. Desenvolvimento Seguro

A solução deve demonstrar:

- Prevenção de XSS (escape, sanitização);
- Validação de dados vindos da UI;
- Armazenamento seguro (evitar expor dados sensíveis);
- Uso consciente de localStorage;
- Isolamento de responsabilidades.

4. Aplicação funcional mínima

Funcionalidades obrigatórias:

- Cadastrar despesa (descrição, valor, categoria);
- Listar despesas;
- Editar despesas;
- Excluir despesas;
- Persistência local.

ENTREGÁVEIS

1. Vídeo demonstrativo

O vídeo deve conter até 15 (quinze) minutos e mostrar:

- A arquitetura de pastas;
- Como cada camada se comunica;
- Funcionamento mínimo da aplicação;
- Demonstração dos componentes do Design System;
- Onde foram aplicados princípios de segurança.

2. Repositório GitHub

O repositório deve conter:

- Código completo.
- README com:
 - Arquitetura utilizada;
 - Justificativa das decisões;
 - Documentação do Design System.

Boa Sorte!